University Question answers set- Module 2

1) Explain in brief the types of CPU schedulers with a diagram. [5M] **Dec 2022**

Ans. Types of Process Schedulers

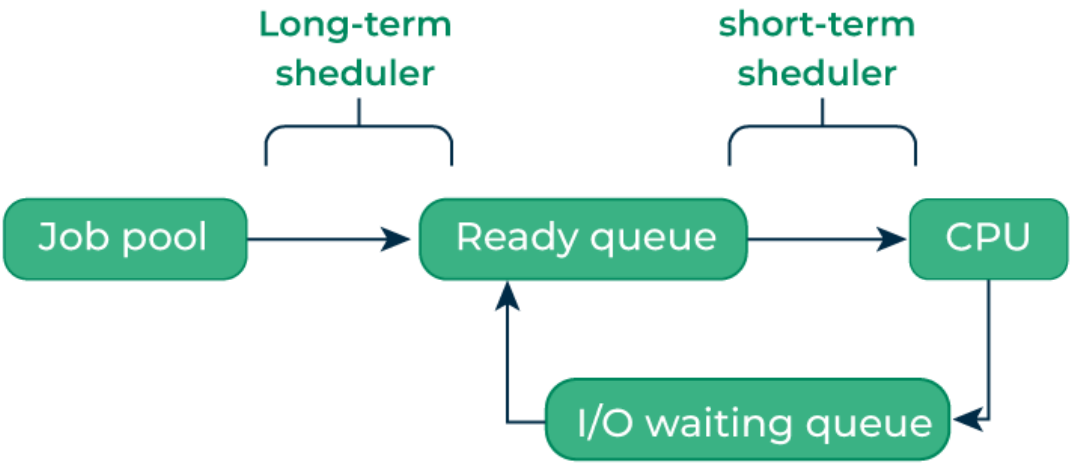
There are three types of process schedulers:

1. Long Term or Job Scheduler

It brings the new process to the 'Ready State'. It controls the Degree of Multi-programming, i.e., the number of processes present in a ready state at any point in time. It is important that the long-term scheduler make a careful selection of both I/O and CPU-bound processes. I/O-bound tasks are which use much of their time in input and output operations while CPU-bound processes are which spend their time on the CPU. The job scheduler increases efficiency by maintaining a balance between the two. They operate at a high level and are typically used in batch-processing systems.

2. Short-Term or CPU Scheduler

It is responsible for selecting one process from the ready state for scheduling it on the running state. Note: Short-term scheduler only selects the process to schedule it doesn't load the process on running. Here is when all the scheduling algorithms are used. The CPU scheduler is responsible for ensuring no starvation due to high burst time processes.



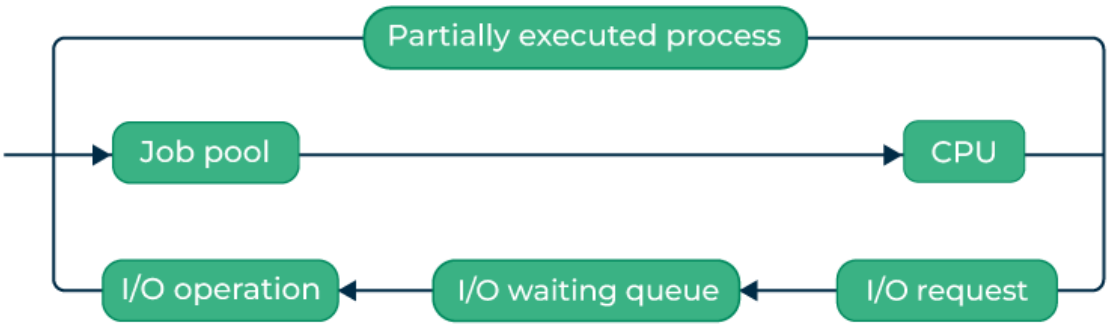
Short Term Scheduler

The dispatcher is responsible for loading the process selected by the Short-term scheduler on the CPU (Ready to Running State) Context switching is done by the dispatcher only. A dispatcher does the following:

- Switching context.
- Switching to user mode.
- Jumping to the proper location in the newly loaded program.

3. Medium-Term Scheduler

It is responsible for suspending and resuming the process. It mainly does swapping (moving processes from main memory to disk and vice versa). Swapping may be necessary to improve the process mix or because a change in memory requirements has overcommitted available memory, requiring memory to be freed up. It is helpful in maintaining a perfect balance between the I/O bound and the CPU bound. It reduces the degree of multiprogramming.



Medium Term Scheduler

2) Define Thread. Mention benefits of Multithreading **[5M]**

Dec 2022/May 2022

Ans.

Thread: A thread is a conscience sequence of instructions that may run in the same parent process as other threads. Also, threads are called as light weight process.

The benefits of multi-threaded programming can be broken down into four major categories:

1. **Responsiveness** – Multithreading in an interactive application may allow a program to continue running even if a part of it is blocked or is performing a lengthy operation, thereby increasing responsiveness to the user. In a non-multi-threaded environment, a server listens to the port for some request and when the request comes, it processes the request and then resume listening to another request. The time taken while processing of request makes other users wait unnecessarily. Instead, a better approach would be to pass the request to a worker thread and continue listening to port. For example, a multi-threaded web browser allow user interaction in one thread

while an video is being loaded in another thread. So instead of waiting for the whole web-page to load the user can continue viewing some portion of the web-page.

2. **Resource Sharing** – Processes may share resources only through techniques such as-
    - Message Passing
    - Shared Memory

    Such techniques must be explicitly organized by programmer. However, threads share the memory and the resources of the process to which they belong by default. The benefit of sharing code and data is that it allows an application to have several threads of activity within same address space.
3. **Economy** – Allocating memory and resources for process creation is a costly job in terms of time and space. Since, threads share memory with the process it belongs, it is more economical to create and context switch threads. Generally, much more time is consumed in creating and managing processes than in threads. In Solaris, for example, creating process is 30 times slower than creating threads and context switching is 5 times slower.
4. **Scalability** – The benefits of multi-programming greatly increase in case of multiprocessor architecture, where threads may be running parallel on multiple processors. If there is only one thread then it is not possible to divide the processes into smaller tasks that different processors can perform. Single threaded process can run only on one processor regardless of how many processors are available. Multi-threading on a multiple CPU machine increases parallelism.
5. **Better Communication System** – To improve the inter-process communication, thread synchronization functions can be used. Also, when need to share huge amounts of data across multiple threads of execution inside the same address space then provides extremely high bandwidth and low communication across the various tasks within the application.
6. **Microprocessor Architecture Utilization** – Every thread could be executed in parallel on a distinct processor which might be considerably amplified in a microprocessor architecture. Multithreading enhances concurrency on a multi-CPU machine. Also, the CPU switches among threads very quickly in a single processor architecture where it creates the illusion of parallelism, but at a particular time only one thread can run.

3) Discuss various CPU scheduling criteria[10M] **Dec 2022**

Ans.

Criteria of CPU Scheduling

CPU utilization

The main objective of any CPU scheduling algorithm is to keep the CPU as busy as possible. Theoretically, CPU utilization can range from 0 to 100 but in a real-time system, it varies from 40 to 90 percent depending on the load upon the system.

Throughput

A measure of the work done by the CPU is the number of processes being executed and completed per unit of time. This is called throughput. The throughput may vary depending on the length or duration of the processes.

Turnaround Time

For a particular process, an important criterion is how long it takes to execute that process. The time elapsed from the time of submission of a process to the time of completion is known as the turnaround time. Turn-around time is the sum of times spent waiting to get into memory, waiting in the ready queue, executing in CPU, and waiting for I/O.

```
Turn Around Time = Completion Time - Arrival Time.
```

Waiting Time

A scheduling algorithm does not affect the time required to complete the process once it starts execution. It only affects the waiting time of a process i.e. time spent by a process waiting in the ready queue.

```
Waiting Time = Turnaround Time - Burst Time.
```

Response Time

In an interactive system, turn-around time is not the best criterion. A process may produce some output fairly early and continue computing new results while previous results are being output to the user. Thus another criterion is the time taken from submission of the process of the request until the first response is produced. This measure is called response time.

```
Response Time = CPU Allocation Time(when the CPU was allocated for the first) - Arrival Time
```

Completion Time

The completion time is the time when the process stops executing, which means that the process has completed its burst time and is completely executed.

Priority

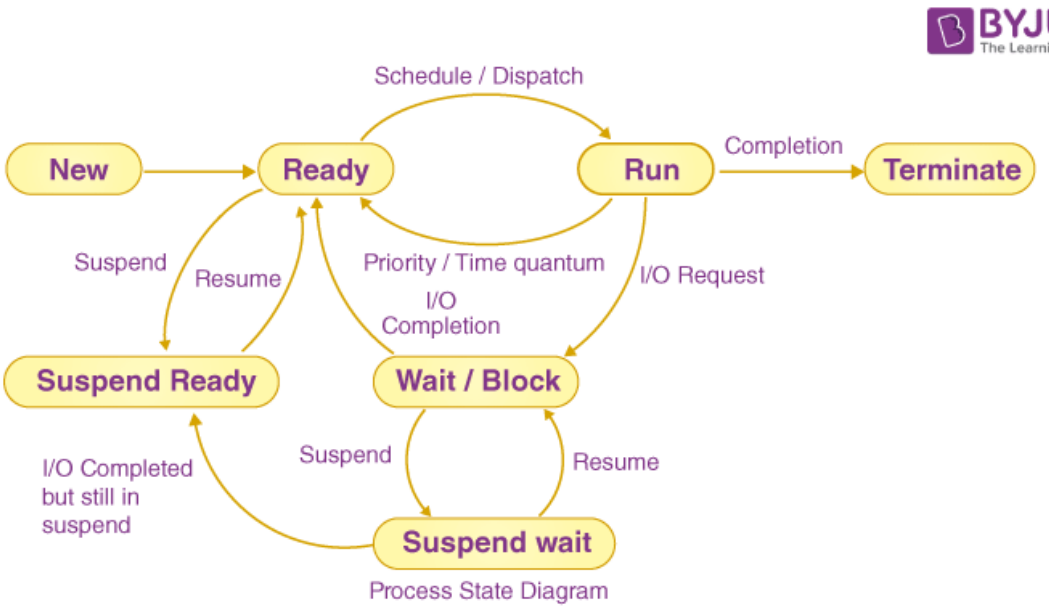
If the operating system assigns priorities to processes, the scheduling mechanism should Favor the higher-priority processes.

4) Explain the Five-State Process State Transition Diagram[10M] **May 2022 / Dec 2022**

Process States

A process has several stages that it passes through from beginning to end. There must be a minimum of five states. Even though during execution, the process could be in one of these states, the names of the states are not standardized. Each process goes through several stages throughout its life cycle.

Process States in Operating System



The states of a process are as follows:

- **New (Create):** In this step, the process is about to be created but not yet created. It is the program that is present in secondary memory that will be picked up by OS to create the process.
- **Ready:** New -> Ready to run. After the creation of a process, the process enters the ready state i.e. the process is loaded into the main memory. The process here is ready to run and is waiting to get the CPU time for its execution. Processes that are ready for execution by the CPU are maintained in a queue called ready queue for ready processes.
- **Run:** The process is chosen from the ready queue by the CPU for execution and the instructions within the process are executed by any one of the available CPU cores.

- **Blocked or Wait:** Whenever the process requests access to I/O or needs input from the user or needs access to a critical region(the lock for which is already acquired) it enters the blocked or waits for the state. The process continues to wait in the main memory and does not require CPU. Once the I/O operation is completed the process goes to the ready state.
- **Terminated or Completed:** Process is killed as well as PCB is deleted. The resources allocated to the process will be released or deallocated.
- **Suspend Ready:** Process that was initially in the ready state but was swapped out of main memory(refer to Virtual Memory topic) and placed onto external storage by the scheduler is said to be in suspend ready state. The process will transition back to a ready state whenever the process is again brought onto the main memory.
- **Suspend wait or suspend blocked:** Similar to suspend ready but uses the process which was performing I/O operation and lack of main memory caused them to move to secondary memory. When work is finished, it may go to suspend ready.

5) Explain Round Robin Algorithm with a suitable example[10M] **Dec 2022**

Ans. Round Robin is a CPU scheduling algorithm where each process is cyclically assigned a fixed time slot. It is the pre-emptive version of the First come First Serve CPU Scheduling algorithm.

- Round Robin CPU Algorithm generally focuses on Time Sharing technique.
- The period of time for which a process or job is allowed to run in a pre-emptive method is called time **quantum**.
- Each process or job present in the ready queue is assigned the CPU for that time quantum, if the execution of the process is completed during that time, then the process will **end** else the process will go back to the **waiting table** and wait for its next turn to complete the execution.

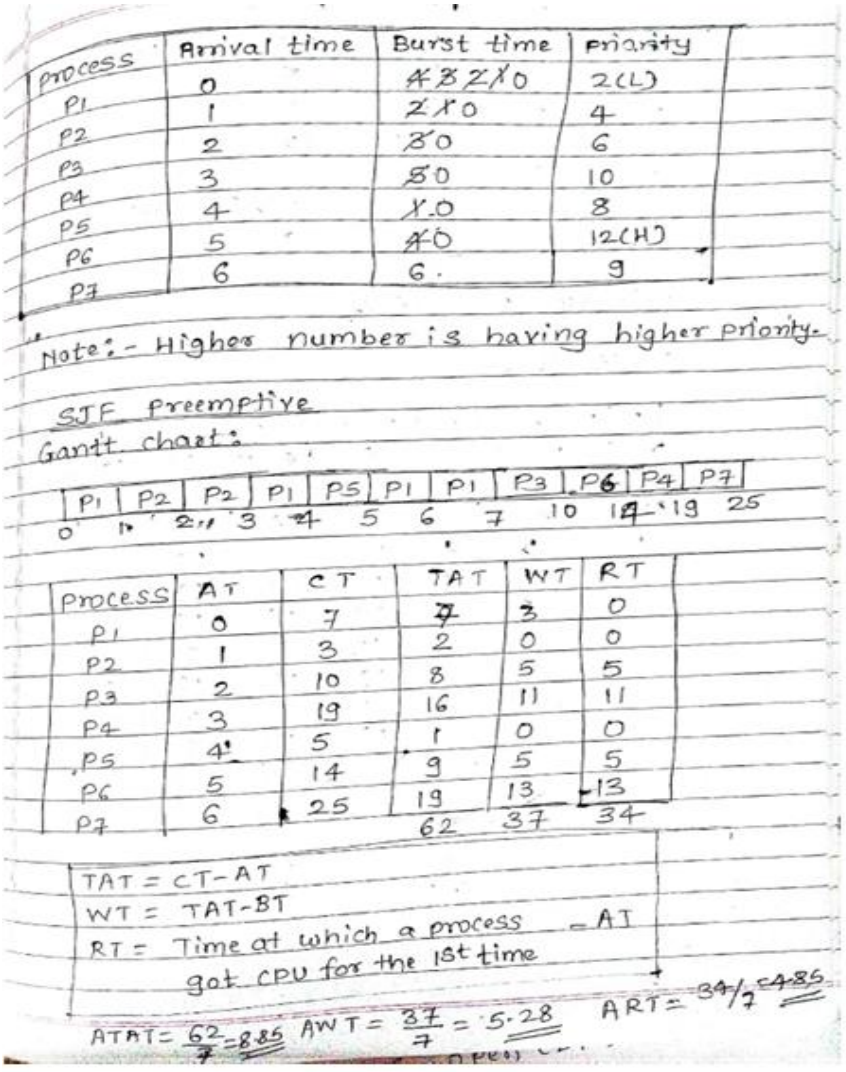
Consider the following set of processes.

Process	Burst Time	Arrival Time	Priority
P1	0	4	2(L)
P2	1	2	4
P3	2	3	6
P4	3	5	10
P5	4	1	8
P6	5	4	12 (H)
P7	6	6	9

Note Higher number is having higher priority.

1. Draw Gantt chart for SJF-Preemptive Scheduling and Preemptive Priority scheduling.
2. Calculate average waiting time, average turnaround time and average response time for this scheduling algorithms.

[10M] **May 2022**



Process	Arrival time	Burst time	priority
P1	0	~~4~~ ~~3~~ 2 ~~1~~ 0	2(L)
P2	1	2 ~~1~~ 0	4
P3	2	~~3~~ 0	6
P4	3	~~5~~ 0	10
P5	4	~~1~~ 0	8
P6	5	~~4~~ 0	12(H)
P7	6	6.	9

Note:- Higher number is having higher priority.

SJF Preemptive

Gantt chart:

P1	P2	P2	P1	P5	P1	P1	P3	P6	P4	P7

0 1 2 3 4 5 6 7 10 14 19 25

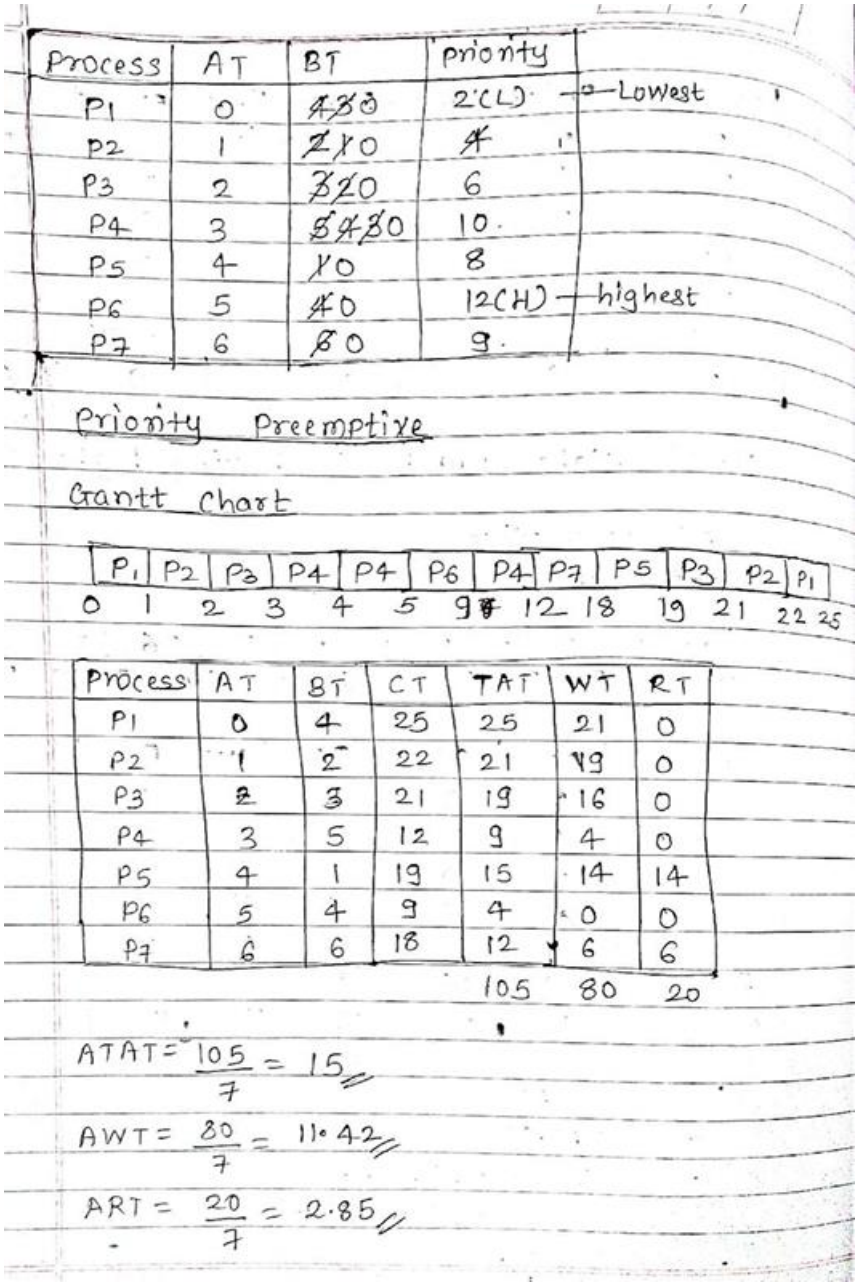
Process	AT	CT	TAT	WT	RT
P1	0	7	~~7~~	3	0
P2	1	3	2	0	0
P3	2	10	8	5	5
P4	3	19	16	11	11
P5	4	5	1	0	0
P6	5	14	9	5	5
P7	6	25	19	13	13
			62	37	34

TAT = CT - AT

WT = TAT - BT

RT = Time at which a process got CPU for the 1st time - AT

ATAT = $\frac{62}{7}$ = 8.85 AWT = $\frac{37}{7}$ = 5.28 ART = $\frac{34}{7}$ = 4.85



Process	AT	BT	priority
P1	0	4	2 (L) — Lowest
P2	1	2	4
P3	2	3	6
P4	3	5	10
P5	4	1	8
P6	5	4	12 (H) — highest
P7	6	6	9

Priority Preemptive

Gantt Chart

P1	P2	P3	P4	P4	P6	P4	P7	P5	P3	P2	P1
0	1	2	3	4	5	9	12	18	19	21	22 25

Process	AT	BT	CT	TAT	WT	RT
P1	0	4	25	25	21	0
P2	1	2	22	21	19	0
P3	2	3	21	19	16	0
P4	3	5	12	9	4	0
P5	4	1	19	15	14	14
P6	5	4	9	4	0	0
P7	6	6	18	12	6	6
				105	80	20

ATAT = $\frac{105}{7} = 15$

AWT = $\frac{80}{7} = 11.42$

ART = $\frac{20}{7} = 2.85$

Explain different types of thread in Operating System [5M] **May 2023**

Types of Threads:

1. **User Level thread (ULT)** – Is implemented in the user level library, they are not created using the system calls. Thread switching does not need to call OS and to cause interrupt to Kernel. Kernel doesn't know about the user level thread and manages them as if they were single-threaded processes.
    - **Advantages of ULT** –
        - Can be implemented on an OS that doesn't support multithreading.
        - Simple representation since thread has only program counter, register set, stack space.

- Simple to create since no intervention of kernel.
- Thread switching is fast since no OS calls need to be made.

- **Limitations of ULT –**
  - No or less co-ordination among the threads and Kernel.
  - If one thread causes a page fault, the entire process blocks.

2. **Kernel Level Thread (KLT) –** Kernel knows and manages the threads. Instead of thread table in each process, the kernel itself has thread table (a master one) that keeps track of all the threads in the system. In addition kernel also maintains the traditional process table to keep track of the processes. OS kernel provides system call to create and manage threads.
   - **Advantages of KLT –**
     - Since kernel has full knowledge about the threads in the system, scheduler may decide to give more time to processes having large number of threads.
     - Good for applications that frequently block.
   - **Limitations of KLT –**
     - Slow and inefficient.
     - It requires thread control block so it is an overhead.

Consider the following set of processes. [10]

Process	Burst Time	Arrival Time
P1	10	0
P2	5	1
P3	2	2

1. Draw Gantt chart for FCFS, SJF(Preemptive) and Round Robin (Quantum=2).
2. Also calculate average waiting time and turnaround time for above scheduling algorithms.

May 2023

process	Burst time	Arrival time
P1	10	0
P2	5	1
P3	2	2

FCFS

TAT = CT - AT
WT = TAT - BT.

P1	P2	P3

0 10 15 17

	CT	TAT	WT
P1	10	10	0
P2	15	14	9
P3	17	15	13

$ATAT = \frac{10+14+15}{3} = \frac{39}{3} = 13$ $AWT = \frac{0+9+13}{3} = \frac{22}{3} = 7.33$

SJF preemptive

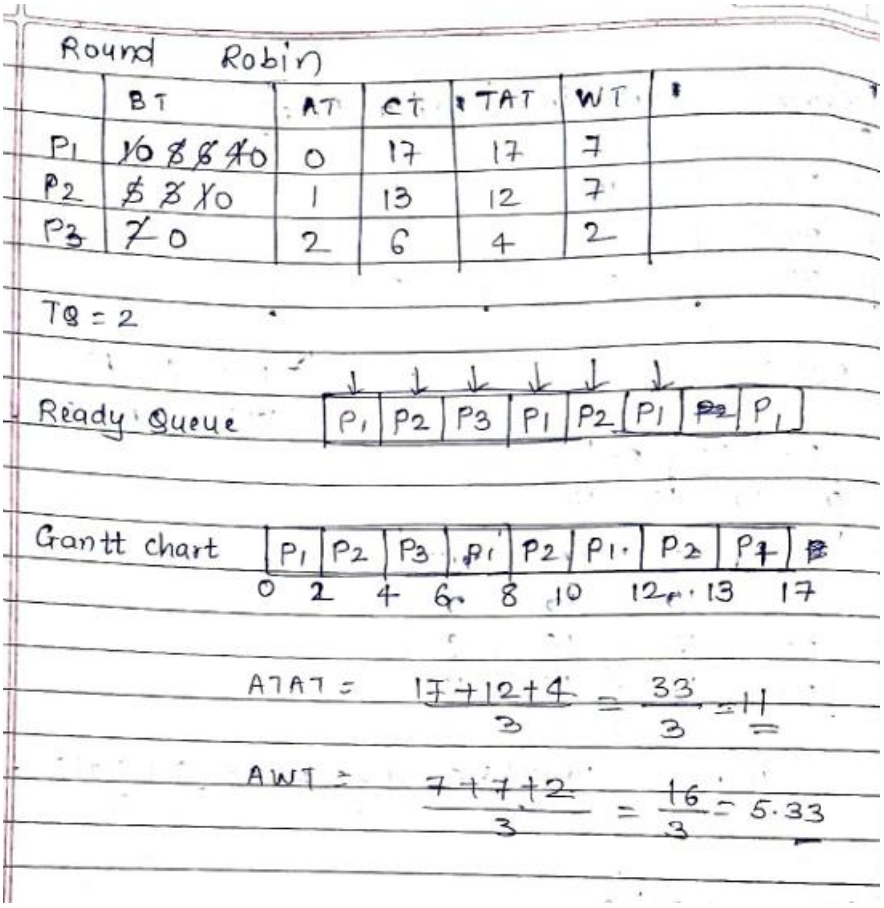
P1	P2	P3	P2	P1

0 1 2 4 8 17

	BT	AT	CT	TAT	WT
P1	~~10~~ 9	0	17	17	7
P2	~~5~~ 4	1	8	7	2
P3	~~2~~ 0	2	4	2	0

$ATAT = \frac{17+7+2}{3} = 8.66$

$AWT = \frac{7+2+0}{3} = 3$

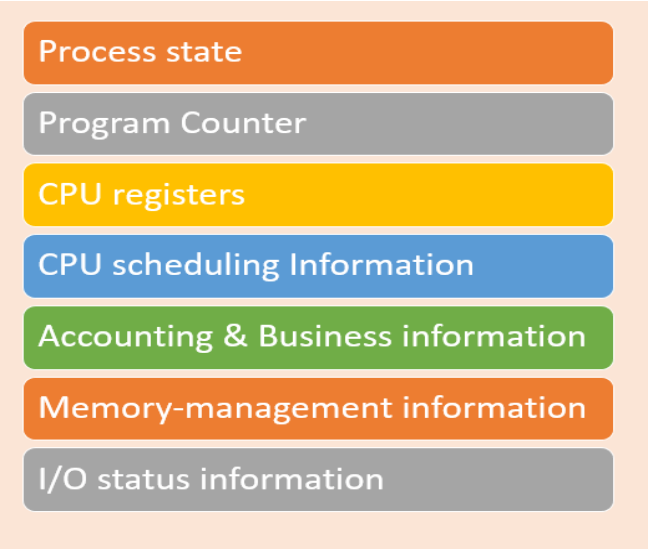


Explain the role of PCB [10M] **May 2023**

Process Control Block

Attributes of Process: **The attributes of a process are also called the context of the process. These attributes design the Process Control Block (PCB).**

Each process is represented in the operating system *Process Control Block PCB -also called Task Control Block*. It is a data structure that is maintained by the Operating System for every process. The PCB should be identified by an integer Process ID (PID). It helps you to store all the information required to keep track of all the running processes.



- **Process state:** A process can be new, ready, running, waiting, etc.
- **Program counter:** The program counter lets you know the address of the next instruction, which should be executed for that process.
- **CPU registers:** This component includes accumulators, index and general-purpose registers, and information of condition code.

- **CPU scheduling information:** This component includes a process priority, pointers for scheduling queues, and various other scheduling parameters.
- **Accounting and business information:** It includes the amount of CPU and time utilities like real time used, job or process numbers, etc.
- **Memory-management information:** This information includes the value of the base and limit registers, the page, or segment tables. This depends on the memory system, which is used by the operating system.
- **I/O status information:** This block includes a list of open files, the list of I/O devices that are allocated to the process, etc.

Additional Points to Consider for Process Control Block (PCB)

- **Interrupt handling:** The PCB also contains information about the interrupts that a process may have generated and how they were handled by the operating system.
- **Context switching:** The process of switching from one process to another is called context switching. The PCB plays a crucial role in context switching by saving the state of the current process and restoring the state of the next process.
- **Real-time systems:** Real-time operating systems may require additional information in the PCB, such as deadlines and priorities, to ensure that time-critical processes are executed in a timely manner.
- **Virtual memory management:** The PCB may contain information about a process's virtual memory management, such as page tables and page fault handling.
- **Inter-process communication:** The PCB can be used to facilitate inter-process communication by storing information about shared resources and communication channels between processes.
- **Fault tolerance:** Some operating systems may use multiple copies of the PCB to provide fault tolerance in case of hardware failures or software errors.